

Sistemas de Operação

(Ano letivo de 2026-2027)

Guiões das aulas práticas

script #03 IPC — File redirection, pipes and fifos

Summary

- Understanding and dealing with redirecting the standard output of a process.
- Using pipes and named pipes as a communication channel between processes.

Previous note

In the code provided, some system calls are not used directly. Instead, equivalent functions provided by the `process.{h,cpp}` library are used. The functions in this library deal internally with error situations, either aborting execution or throwing exceptions, thus releasing the programmer of doing so. This library will be available during the practical exams.

Exercises

Exercise 1 Redirecting the standard output (stdout) of a process.

(a) ***The `dup2` system call.***

- See the manual for a description of the `dup2` system call (`man dup2`).
- Read the `redirect.cpp` source code, create the `redirect` executable (`make redirect`[1]), and run it.
- **Questions:** What happened to the program's second `printf` statement? Was it executed or not? If it was, where did the printed message go to?
- Edit the `my.file` file which has since been created in the working directory. Change its content by inserting your own text with at least 40 characters. Run the `redirect` program again.
- **Question:** Explain the change that occurred to the file's content after the previous execution. Was that what you expected?

(b) ***Training exercise***

An exercise from the previous lesson involves programs `fork4.cpp` and `child.cpp`, both sending messaged to the standard output. Without changing the `printf` statements of any of them, change program (`fork4.cpp`), and only it, such that the `printf`s of program `child.cpp` go to a file whose name is given to `fork4.cpp` as a command line argument.

Continue in next page

[1] First, you need to go to the `support` folder and run `make` there.

Exercise 2 Creating and using a pipe as a communication channel between two processes.

A pipe is a unidirectional data transfer channel that can be used to facilitate communication between directly related processes (parent-child or child-child, for example). A pipe can be created by calling system call `pipe` (see `man 2 pipe`). As you can see, it takes as a parameter a pointer to an array of two integers, used to return two file descriptors referring to the read and write ends of the pipe. To read and write, using a file descriptor directly, one should use system calls `read` and `write`, respectively. Alternatively, one can use the `fdopen` function to allow the use of the `stdio` library.

(a) ***Creating a pipe.***

- Read the `pipe1.cpp` source code, create the `pipe1` executable (`make pipe1`), and run it.
- **Question:** What led to the modification of the contents of the `buf` array?
- Edit line 32 of the `pipe1.cpp` file, eliminating the `+1` in the expression assigned to the `sz` variable. Rebuild the executable file and run it in the new version. Try to interpret the changes.

(b) ***Communication between a parent and a child processes using a pipe.***

- See the manual for a description of the `sort` command (`man sort`).
- Read the `pipe2.cpp` source code, create the `pipe2` executable (`make pipe2`), and run it.
- Execute the following sequence of commands:

```
ls -l > /tmp/zzz
sort -n -k 5 /tmp/zzz
```

- Execute the following command

```
ls -l | sort -n -k 5
```

- **Question:** Are there differences between the outputs of the 3 previous cases? Why or why not?

(c) ***Training exercise***

Create new program, `pipe3.cpp`, such that:

- The second `exec` (the one related to the `sort`) also happens in a child process.
- The parent process waits for the termination of both children before quit.

Continue in next page

Exercise 3 Communication between two processes using a named pipe (FIFO)

A FIFO, or named pipe, is a particular type of pipe that is accessible through the file system (in fact, it constitutes a special type of file). This means that, unlike a conventional pipe, its existence is independent of the processes that use it, enabling communication between processes that are not directly related. See the manual for a description of the FIFO special file (`man fifo`). A FIFO can be created using command `mkfifo` (see `man mkfifo`).

(a) ***Creating a FIFO.***

- Create a FIFO in your working directory called `my.fifo`

  ```
  mkfifo my.fifo
  ```

  List the directory

  ```
  ls -l
  ```

  and see what happens.

(b) ***Communicating using a FIFO.***

- Analyze the code of the programs described by the files `sender.cpp` and `receiver.cpp` that implement a mechanism for transferring messages from one terminal to another.
- Create the sender (`make server`) and receiver (`make receiver`) executables and run them in separate terminals. Launch receiver first, then sender.
- **Question:** Why does the "Received through fifo" message only appear on the receiver terminal after launching the sender?
- **Question:** Try to explain how the message transfer mechanism works between the sender process and the receiver process.
- **Question:** Press the CTRL-D key combination in the terminal where you launched the sender process. Not only did the sender process terminate, but also the receiver process. Why?

(c) ***Training exercise***

Modify the `sender.cpp` and `receiver.cpp` files to allow bidirectional communication.
Suggestion: use two FIFOs.